

Forecast-Driven MPC for Battery Dispatch

Diagnosing why the controller lost to a dumb plan, fixing it, and explaining the residual gap through the physics of price spikes

An investigation report — grian / SA1 30-minute common window (Jul 2025 – Jun 2026)

Battery: 100 MW / 200 MWh, 85% round-trip, 2 cycles/day

Abstract. A forecast-driven model-predictive controller (MPC) for battery arbitrage scored roughly **half** the capture of a naive open-loop plan — an impossibility for a correct controller, since MPC has strictly more information. We trace the failure to a single modelling choice (*observe_present*) that made the optimiser trade high-frequency forecast noise, fix it with a spike-gated variant that beats open-loop on every regime, and then explain a subtler residual effect: even with that bug removed, re-forecasting every interval underperforms committing to a day-ahead plan. The cause is not the controller but the **physics of the price signal**: the forecast is a strong daily-shape prior that cannot anticipate spikes, and annual value is dominated by a handful of unpredictable mega-spike days. A committed plan stays charged at peak times and catches those spikes by positional luck; a churning controller does not. This report documents the full chain of evidence and closes with the exact training recipe of the forecasting model under test.

1. Setup: the deliverable and the metric

The deliverable is a **forecast-driven MPC** that dispatches a grid battery in the Australian National Electricity Market (NEM), region SA1. Every result here is on one fixed held-out window — the **30-minute common window**, 1 Jul 2025 to 30 Jun 2026 — using prices that are the mean of the six 5-minute settlement intervals in each half hour (the true NEM 30-minute price).

Metric — capture ratio. We score a controller by the revenue it earns as a fraction of a **perfect-foresight oracle** that solves the same dispatch LP on the realised prices: $\text{capture} = (\text{controller revenue}) / (\text{oracle revenue})$, in $[0, 1]$. It is a *decision* metric, not a forecast-accuracy metric — a distinction that turns out to drive everything.

Open-loop vs MPC. Open-loop forecasts the whole day once, solves the LP, and commits — it executes all 48 half-hourly actions regardless of what happens. MPC (receding-horizon control) re-observes the true state, re-forecasts, and *re-solves* the LP every few intervals, executing only the first action before rolling forward. Two cadences parameterise it: *reforecast_every* (how often the forecast is regenerated) and *resolve_every* (how often the LP is re-solved). Open-loop is the special case where both equal the horizon.

The load-bearing theorem. By the Bellman principle of optimality, re-solving an optimal-control problem from the current state over the remaining horizon reproduces the tail of the original optimal plan. MPC therefore has strictly more information than open-loop and **can never score below it**. So an MPC that earns half of open-loop is a **bug**, not a fact of nature. That single deduction drove the entire investigation.

2. Part I — The controller bug: *observe_present*

The symptom: on the full window the naive 30-minute MPC captured ~0.25 while open-loop captured ~0.54 and the simple autoregressive point forecast (the best cheap baseline) reached 0.543.

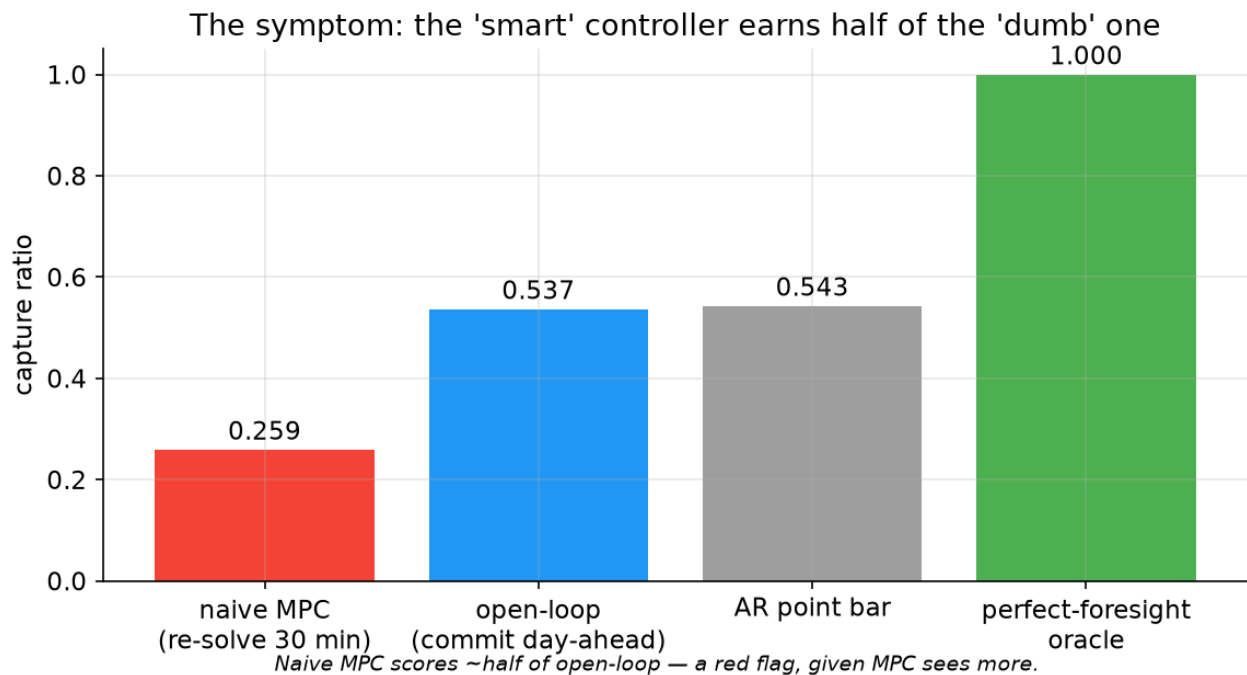


Figure 1. The symptom: the receding-horizon controller earns roughly half of the commit-and-forget plan — impossible for a correct controller.

Decomposition. The MPC changed two things versus open-loop: it re-forecasts and it re-solves more often. Holding one fixed while varying the other isolates the cause. With the forecast held fixed (reforecast=48), simply re-solving more often collapsed capture (0.54 → 0.21); re-forecasting more often was roughly neutral. The damage lives in the **re-solve** path — overturning the project's earlier 'reforecast churn' explanation.

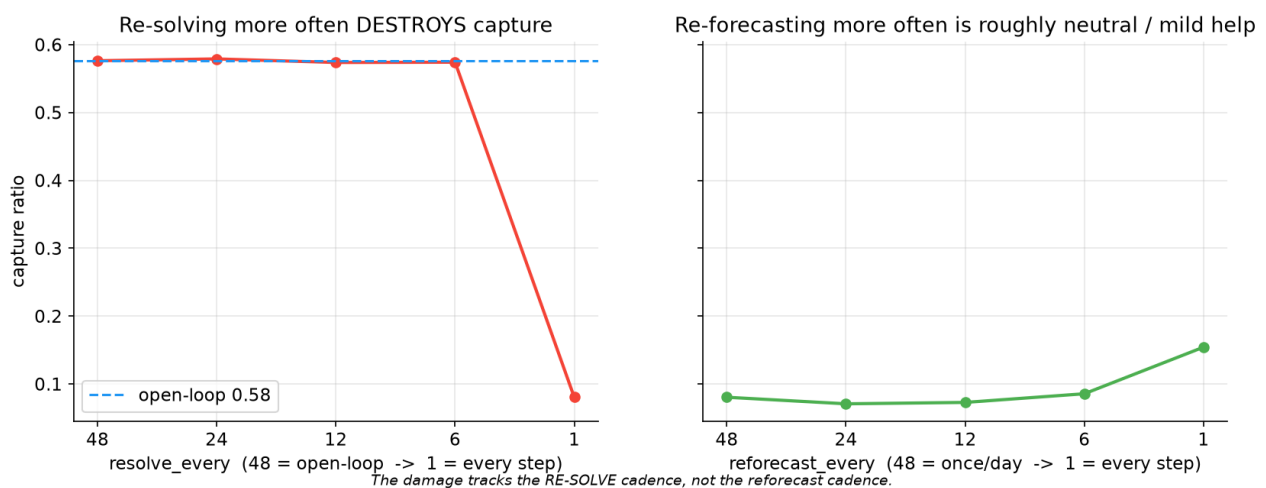


Figure 2. Capture vs the two cadences. The loss tracks re-solve frequency (left), not reforecast frequency (right).

The bug. By Bellman, re-solving on a fixed forecast must reproduce open-loop. It did — but only with *observe_present* turned **off**. That flag, on by default, pins the plan's first step to the *actual* current price (a physically reasonable idea: at decision time you know the current price). But step 0 is then the true

price while steps 1–47 remain the *smoothed* forecast, so the LP sees a discontinuity between the sharp present and the muted future and arbitrages it — trading the high-frequency forecast **residual** (mean-reversion noise) every re-solve. Harmless when it fires twice a day; lethal at every step.

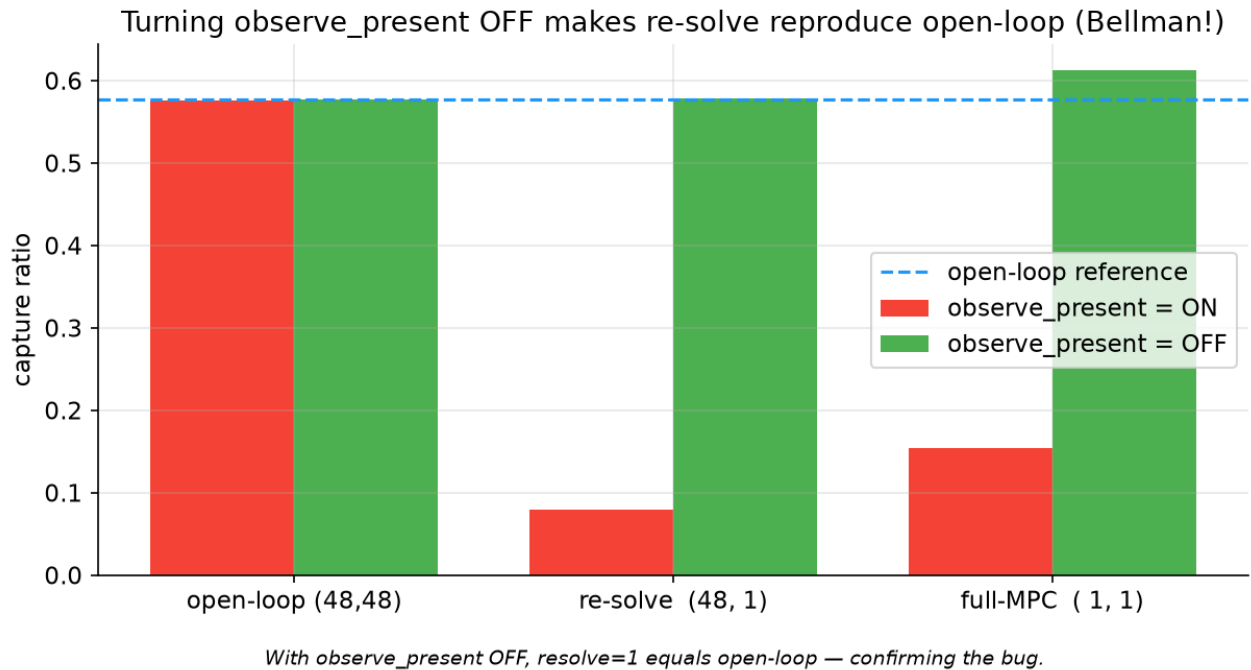


Figure 3. With observe_present OFF, re-solving every step reproduces open-loop exactly (Bellman confirmed); ON, it collapses. The bug is localised.

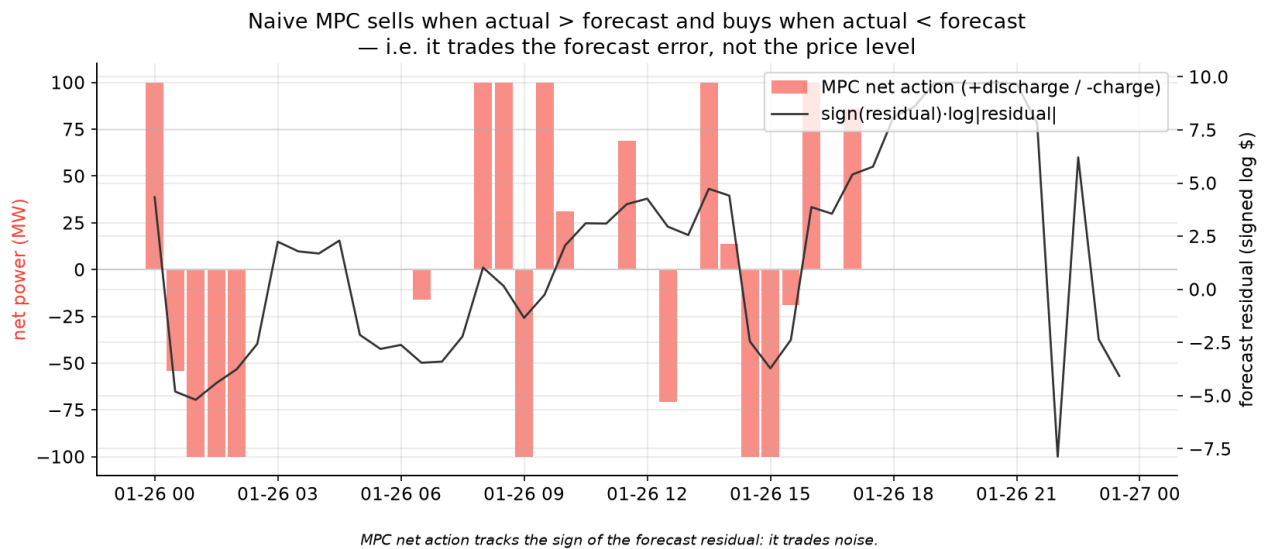


Figure 4. Mechanism: the controller's net action tracks the sign of the forecast residual — it buys the dips and sells the bumps of noise.

The fix — spike-gated observe. We do not discard the present: knowing the true price is genuinely valuable *when it is a real spike*, the one event the forecast cannot see (Part II). The fix (*observe_gate*) pins the present into the plan only when it exceeds a threshold (\$/MWh); otherwise it trusts the forecast. On ordinary intervals the controller is then Bellman-identical to open-loop; on a genuine spike it discharges into the real price.

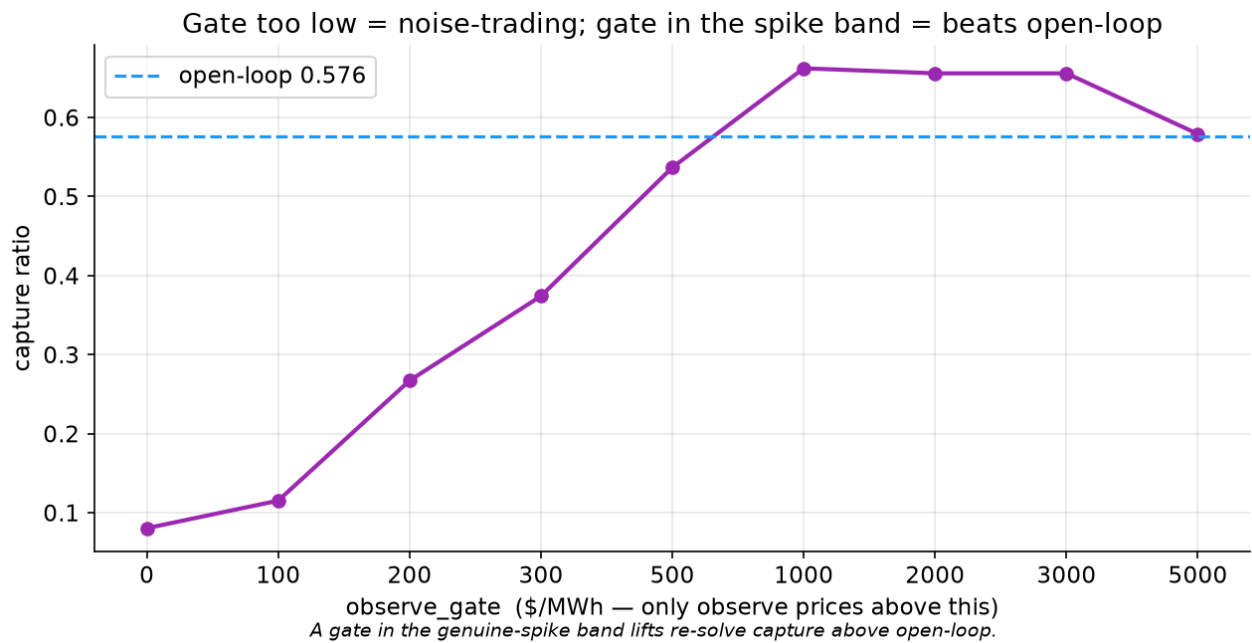


Figure 5. Capture vs the gate. At gate 0 (always observe, the bug) capture is on the floor; in the genuine-spike band (~\$2–3k) it rises above open-loop.

Cross-validation. Tuned on one window, a knob can over-fit. The gate is not fitted noise: it beats-or-ties open-loop on every regime — a large win in the spiky month, an exact tie in the calm month (with no prices above the gate it is provably open-loop), and a solid win on the full year, above the AR bar.

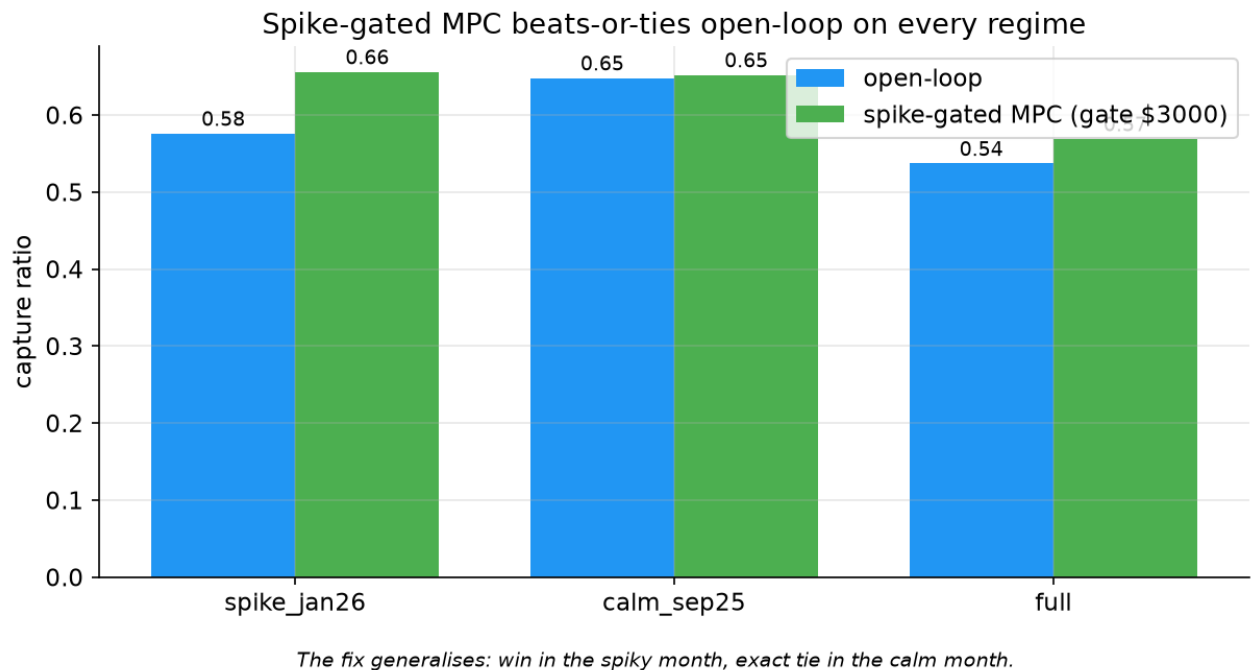


Figure 6. Spike-gated MPC beats-or-ties open-loop on the spiky month, the calm month, and the full year. The mechanism is causal, so it generalises.

3. Part II — What the forecast actually does (why spikes are unforecastable)

The gate presumes spikes cannot be forecast. This part establishes that empirically, and in doing so explains the forecaster's behaviour.

Flat in lead. Forecasting one fixed evening window at several lead times, the curves overlap and none catch the real peak: a forecast made 18 h ahead is as good as one made 30 min ahead. For its forward horizon — the part the controller plans on — the model barely conditions on the present.

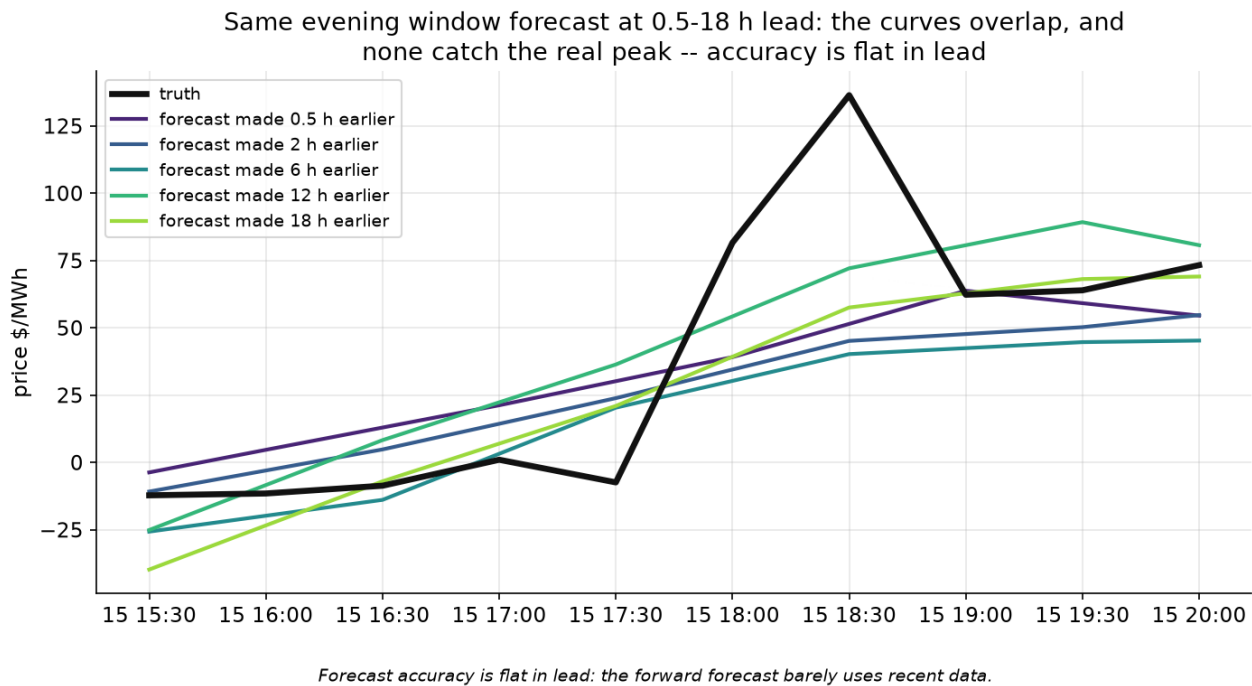


Figure 7. The same window forecast at 0.5–18 h lead. Accuracy is flat in lead: the forward forecast is a typical-day curve.

It can react but not anticipate. A rolling forecast that explicitly sees the previous interval $GT(t-1)$ tracks calm stretches but at each spike lags one step and dampens hard. A spike at t is not predictable from $t-1$ — it is driven by unit trips and bidding, not the recent price path. The best any forecast can do at a spike is 'roughly persist or revert', which under-shoots. This is an information limit, not a weak model.

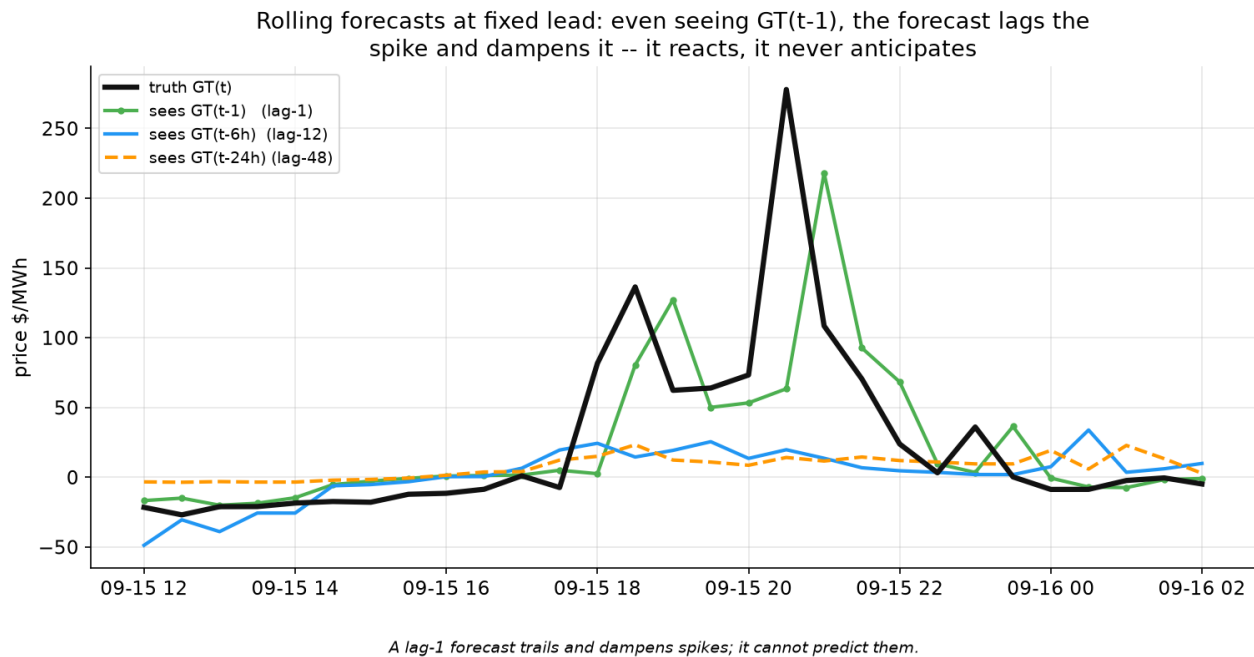


Figure 8. Rolling fixed-lead forecasts. Even seeing $GT(t-1)$ (green), the forecast trails and dampens the spikes — reacting, never anticipating.

Prior on shape, knob on level. Offsetting the recent input by a constant bias slides the forecast's *level* up or down but leaves its *shape* intact — a strong daily-shape prior (from same-time-yesterday, calendar). Sensitivity is ~ 0.65 \$ of level per \$ of input.

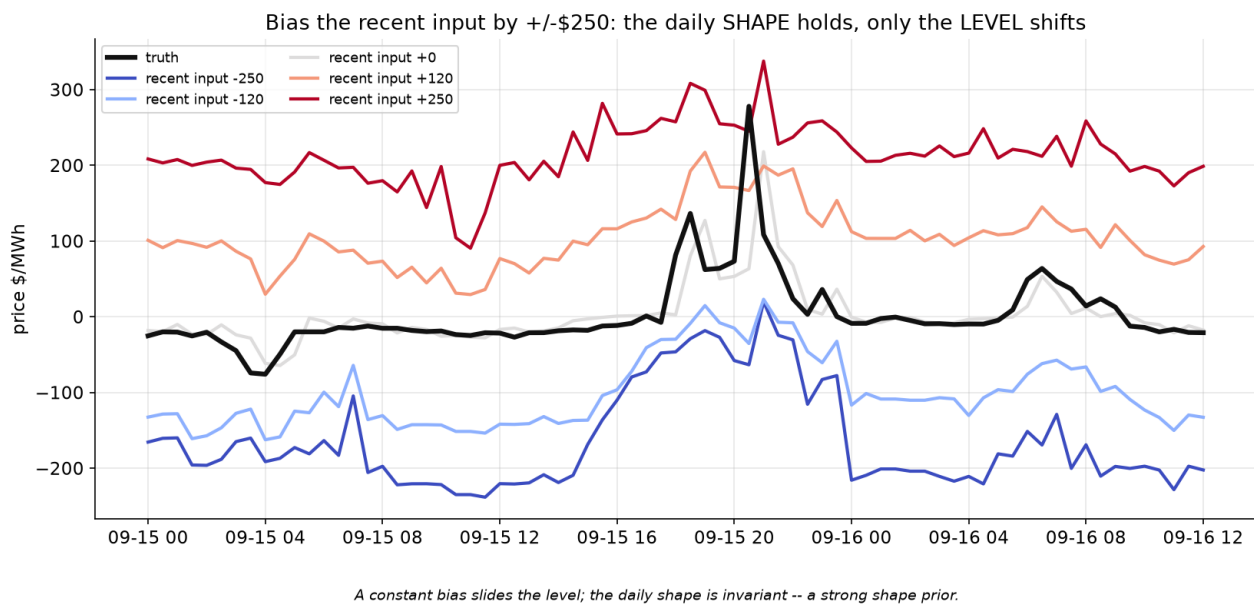


Figure 9. Biasing the recent input by $\pm \$250$: the daily shape holds; only the level shifts. The prior is on the shape.

Zero-mean per-interval noise, by contrast, passes through ($\sim 0.85x$) and jitters the forecast — the level genuinely tracks the recent value; the prior is on the shape, not on noise-robustness. Together: forecast $\approx$ daily-shape-prior + level, with level $\approx 0.7 \times$ recent input.

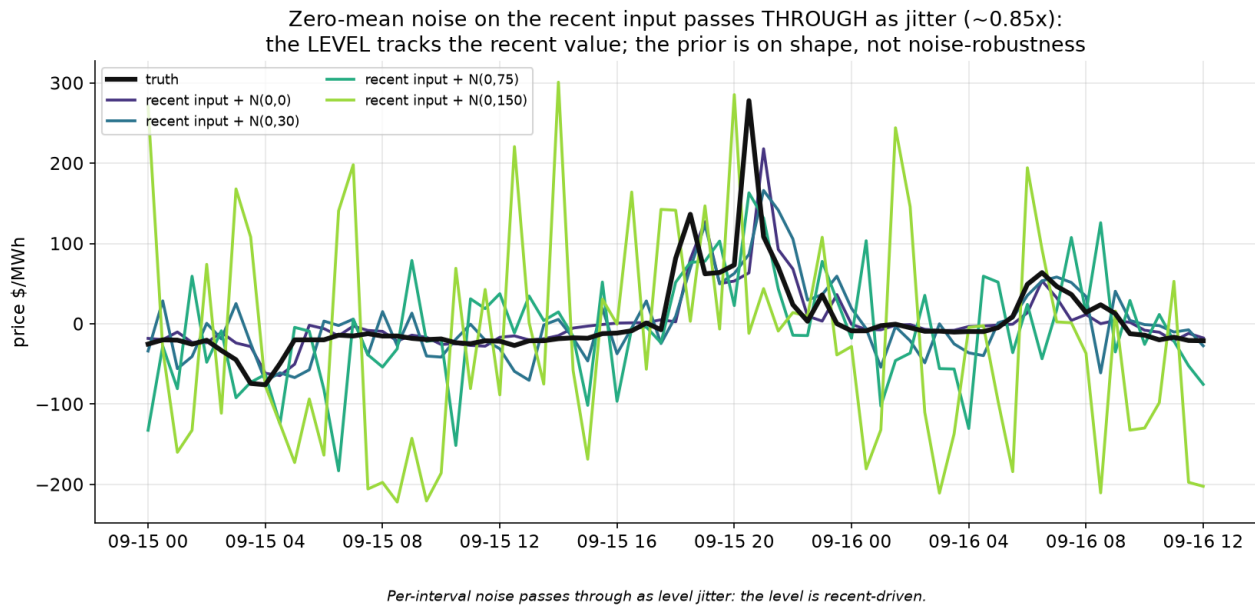


Figure 10. Gaussian noise on the recent input passes through as level jitter: the level is recent-driven, the shape is a prior.

Shrinkage toward the prior. Per interval, comparing the input's deviation from the prior ($X-P$) to the prediction's ($F-P$): the prediction is on the same side of the prior 94% of the time and smaller in magnitude 67% — it pulls the input toward the prior, most aggressively for spikes (slope 0.83 in the body, 0.12 in the tail). Crucially the *truth* reverts the same way: a \$10k input deviation is followed by only ~\$600 of actual deviation. So the shrinkage is mostly correct — spikes are one-interval events that genuinely revert.



Figure 11. Left: per-interval prediction vs input deviation from the prior (dashed = follow input, orange = pure prior). Right: the transfer curve with the truth — the model shrinks spikes toward the prior, and so does reality.

Consequence. A spike is a *shape* deviation; the forecast can only shift the level, not bend the shape, so it structurally cannot produce a spike that is not already in the prior shape. The only moment the true present carries decisive, non-prior information is *during* a spike — which is exactly what the gate in Part I exploits.

4. Part III — Why fresh-reforecast MPC still loses to a committed plan

One effect remained unexplained. Even with *observe_present* off, re-forecasting every interval (B) underperformed committing to the day-start forecast (A): 0.437 vs 0.539 on the full year. Both re-solve every step, so the difference is purely from forecast updates.

The LP returns a plan, not an action. The optimiser produces a full 48-step (24 h) charge/discharge schedule; the controller commits only the first step and re-solves. So we can watch the planned action for a fixed future interval across the successive re-solves that plan it. With a fixed forecast the plan is stable (churn ~0.08 MW, Bellman); with fresh forecasts it oscillates between 0 and full 100 MW discharge (churn ~28 MW).

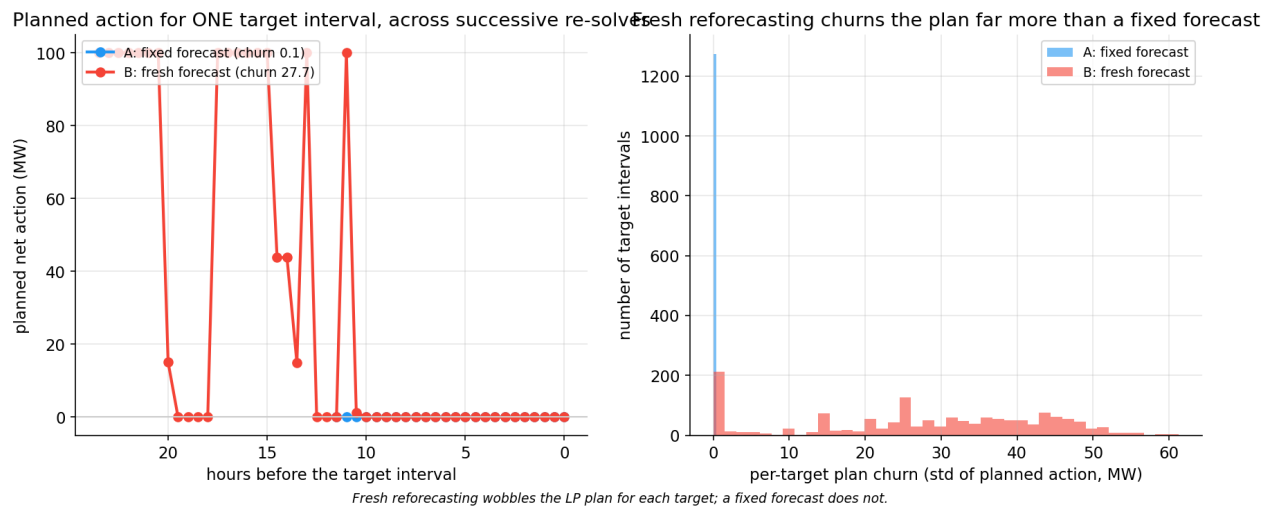


Figure 12. Left: the planned action for one target interval across re-solves — fixed forecast (blue) is flat, fresh forecast (red) wobbles wildly. Right: the distribution of that churn across all targets.

But churn is not intrinsically harmful. A per-month breakdown showed fresh forecasting *helping* in nine months and *hurting* catastrophically in three (Jul 2025, Jan 2026, and Jun 2026, where capture fell to 0.065). The three loss months are precisely those whose value is concentrated in a few mega-spike days.

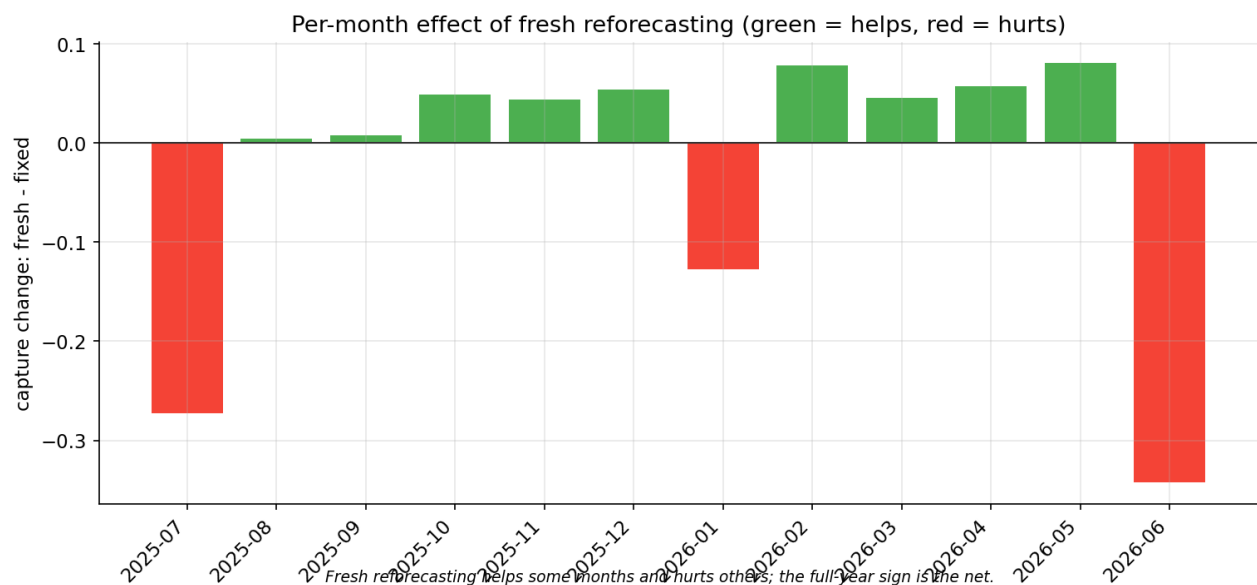
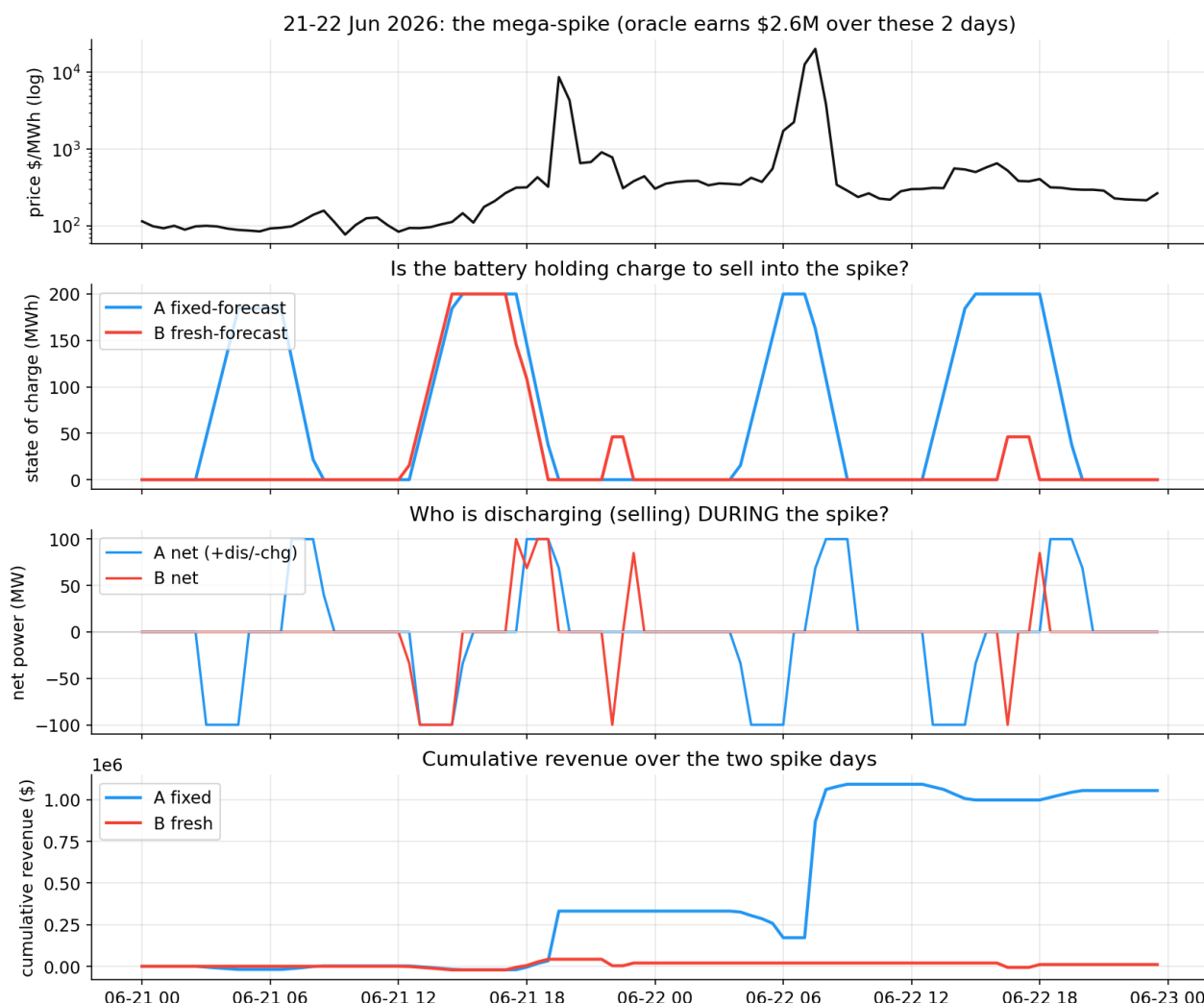


Figure 13. Per-month effect of fresh reforecasting. It helps most months but collapses in three spike-dominated ones, which dominate the annual mean.

The mechanism, on the worst days. June 21–22 were two extreme spike days worth **82% of the month's oracle value** (\$2.6M of \$3.2M). On them the committed controller discharged 100 MW into the spikes and earned ~\$1.2M; the fresh-reforecast controller sat at zero state of charge and **idle**, earning \$0 on the day the oracle earned \$1.9M.



A holds charge and discharges into the spike; B churns and misses it.

Figure 14. June 21–22 mega-spike. A (fixed, blue) runs a charge-hold-discharge cycle, so it is charged when the spikes hit and dumps 100 MW into them. B (fresh, red) churns, stays near-empty, and misses them entirely.

The committed plan is not smart about the spike — it cannot predict it. It runs the daily arbitrage (charge, hold, discharge at the forecast peak), which keeps the battery loaded at high-demand times, and scarcity spikes occur at exactly those times. So it catches them by **positional luck**. The churning controller never commits to 'charge fully and hold' — it dithers with small trades, stays near-empty, and is unloaded when the spike arrives.

This generalises. Across all twelve months the correlation between a month's spike concentration (share of oracle value in its top-2 days) and the fresh-vs-fixed capture gap is **-0.93**. The more a month's value is concentrated in unpredictable mega-spikes, the more a committed plan beats a churning one.

Month (worst gap first)	Top-2-day share of oracle	B - A (fresh - fixed)
2026-06	0.82	-0.34
2025-07	0.60	-0.27
2026-01	0.70	-0.13
9 other months	0.12 - 0.23	+0.01 to +0.08
correlation		-0.93

5. Synthesis and implications

Two distinct churn failures. (i) *observe_present* churned the LP's **inputs** — pinning the sharp present against a smooth forecast so the optimiser traded noise. (ii) Fresh re-forecasting churns the **plan** — re-levelling the schedule every interval so the battery never commits to holding charge for the peak. Both are cured by the same design: a committed day-ahead plan, overridden only by a genuine, observed spike (the gate).

Why a committed plan is a good baseline. It is a *positional hedge* against unpredictable spikes: by always holding charge for the forecast peak, it is loaded when scarcity strikes — which is when it matters, because a few spike days dominate annual value. The spike-gated MPC keeps that hedge and adds the one thing open-loop lacks: it reacts to the spike once it is observed, beating even the committed plan's lucky positioning.

Implication for evaluation. Because a handful of spike days dominate capture, a single annual capture number is extremely high-variance — it is mostly 'did you catch the two big days'. Robust model comparison should report **per-month** or otherwise spike-balanced capture, not one pooled ratio. This directly motivates the balanced-metric due-diligence that follows this report.

Appendix A — Precise training recipe of the model under test

Every result above uses one forecasting model, **lightgbm_qmean**, replayed through the dispatch harness. This appendix specifies it exactly so the experiments are reproducible.

A.1 Data

Region SA1. Source series data/processed/SA1_30min_sim.parquet, the mean of the six 5-minute NEM settlement prices in each half hour (the true 30-minute settlement price). AEMO timestamps are interval-ending and are shifted back by one interval on load. Columns used: price (\$/MWh, the target) and demand (MW, exogenous).

A.2 Target transform

The model targets the inverse hyperbolic sine of price, $y = \operatorname{arcsinh}(\text{price})$, and inverts with $\sinh$ before scoring, so errors and revenue are in dollars. asinh compresses the heavy right tail while preserving sign and near-linearity around zero (unlike $\log_{1p}$, it handles the negative prices common in SA1).

A.3 Model structure — direct multi-step quantile GBM

One LightGBM regressor per (horizon step, quantile). Horizon = 48 steps (24 h at 30 min). Steps are strided at the hourly rate: `step_stride = 2`, i.e. boosters at steps 0,2,4,...,46 plus 47; intermediate steps are linearly interpolated. Quantile levels τ in $[0.05, 0.5, 0.9, 0.98]$. This is a *direct* strategy (each step predicted from the origin's features), not iterative — the model never conditions on its own predictions.

The point forecast the MPC consumes is the quantile-integrated **mean** in dollar space (quantiles inverted to dollars, sorted to repair crossings, then integrated with midpoint trapezoidal weights) — the correct point estimate for a linear revenue objective on a right-skewed price, and the quantity the LP needs.

A.4 LightGBM hyperparameters

```
objective      = 'quantile'    (alpha = tau, one booster per quantile)
n_estimators    = 150
learning_rate  = 0.05
num_leaves     = 31
min_child_samples = 20
subsample      = 0.8
colsample_bytree = 0.8
n_jobs         = -1,  verbosity = -1,  seed = 42
```

A.5 Feature set (30 features from `grian.sim.features.build_features`)

```
price lags      : price_lag_48, price_lag_96, price_lag_336    (1d, 2d, 7d)
price rolling   : price_{rmean,rstd,rmin,rmax}_{1h,6h,24h}    (12 cols)
price momentum  : price_ret_{1h,6h,24h}, price_direction_1h
intraday profile : intraday_profile_mean, intraday_deviation
calendar        : hour, day_of_week, month, is_weekend, hour_x_dow
demand (exogenous): demand, demand_rmean_6h, demand_rmean_24h, demand_rstd_24h
```

Note: shortest price lag is 48 intervals (24 h). There is NO lag-1 feature; recent conditioning enters only via the 1-hour rolling stats / returns. This is why the forward forecast is a smooth daily-shape prior (Part II). Weather (ERA5) and spike-precursor (scarcity) features exist in the library but are OFF for this model.

A.6 Rolling-origin backtest / refit

Walk-forward with an embargo the length of the horizon (no leakage; a unit test proves that injecting a future value degrades the score). Training lookback = 548 days; the model is refit every **28 days** (monthly) on all observed history up to the refit. Forecasts are recorded once at every 30-minute origin into a cached 'fan', over which dispatch variants are replayed — so the experiments in this report share one frozen set of forecasts.

A.7 Battery and dispatch

```
power_mw      = 100      capacity   = power x duration = 200 MWh
duration_hours = 2        efficiency = 0.85 round-trip (sqrt each way)
max_cycles/day = 2        daily discharge budget = 2 x 200 = 400 MWh
resolution     = 30 min   horizon    = 48 steps (day-ahead)
LP solver      = HiGHS (linprog); dt-hours = 0.5; per-day throughput caps.
```

A.8 Oracle and capture metric

The oracle solves the same dispatch LP on the realised prices in weekly blocks with state-of-charge pinned to zero at block ends (closed-cycle, ~0.2% conservative vs a rolling perfect-foresight controller). Capture ratio over any window = $\text{sum}(\text{model daily revenue}) / \text{sum}(\text{oracle daily revenue})$. The full-window oracle earns \$25.18M; its top-10 days are ~45% of that value.

A.9 Controller configurations used in this report

```
open-loop      : reforecast_every = 48, resolve_every = 48
A (fixed fc)   : reforecast_every = 48, resolve_every = 1,  observe_present=off
B (fresh fc)   : reforecast_every = 1,  resolve_every = 1,  observe_present=off
naive MPC      : reforecast_every = 1,  resolve_every = 1,  observe_present=ON
spike-gated fix: reforecast_every = 48 (daily), resolve_every = 1,
                  observe_present=ON, observe_gate = $3000 (executor 'mpc_spike')
dispatch_mode  = point (quantile-integrated mean of the fan).
```

A.10 The battery-arbitrage LP (grian.sim.lp.solve_lp)

Over a planning horizon of T intervals with forecast prices p_t (\$/MWh) and interval length $\Delta t = 0.5$ h, the linear program chooses charge c_t , discharge d_t (MW) and state of charge s_t (MWh):

```
maximise  sum_t p_t * (d_t - c_t) * dt          [revenue, $]
subject to
  s_t = s_{t-1} + eta*c_t*dt - (1/eta)*d_t*dt    [SOC dynamics]
  s_0 = soc0, eta = sqrt(efficiency) = sqrt(0.85)
  0 <= c_t, d_t <= P                          (P = 100 MW power limit)
  0 <= s_t <= S_bar                           (S_bar = 200 MWh capacity)
  for each calendar-day segment [a,b) in the horizon:
    sum_{t in [a,b)} d_t*dt <= B_day           [throughput/cycle cap]
  (B_day = remaining daily budget for today, full 400 MWh for later days)
```

It is a pure LP (convex, exactly solved by HiGHS via `scipy.linprog`); there is no approximation in the optimiser. The efficiency is split as $\sqrt{\eta}$ on each leg. The objective is linear in price, which is why the point forecast fed to it must be the conditional *mean*, not the median (A.12).

A.11 The receding-horizon controller (grian.sim.mpc.simulate_region_mpc)

The controller walks the test window, re-forecasting and re-solving on their cadences and executing only the first block before rolling forward:

```

soc <- 0
for pos in test_window step resolve_every:
  if pos crosses a refit boundary (every refit_days):
    refit model on all history up to pos
  if pos - forecast_origin >= reforecast_every:      # re-forecast
    fan      <- model.predict_fan(history up to pos) # dollar quantile paths
    forecast <- quantile_integrated_mean(fan)        # point path (A.12)
    forecast_origin <- pos
  plan <- forecast[pos - forecast_origin :]          # remaining horizon
  if observe_present and actual_price[pos] >= observe_gate: # spike-gated
    plan[0] <- actual_price[pos]                    # pin the true present
  budgets <- per-day throughput caps from remaining daily budget
  (charge, discharge) <- solve_lp(plan, soc0=soc, throughput_budgets=budgets)
  for i in 0 .. resolve_every-1:                    # execute first block
    clamp (charge_i, discharge_i) to true soc & remaining budget
    revenue_i <- actual_price[pos+i] * (discharge_i - charge_i) * dt
    update soc, remaining daily budget; log the interval

```

Only the first *resolve_every* actions of each plan are executed; the LP returns the full 48-step schedule but the rest is discarded and re-planned. Execution is re-clamped for feasibility against the *true* state of charge and remaining budget (`clamp_action`), and revenue is computed only from clamped actions. Open-loop is the special case `reforecast_every = resolve_every = 48`; the reported fix is daily re-forecast, re-solve every interval, with the spike gate. At 30 min the horizon (48) is short enough that no telescoping is applied.

A.12 From quantile fan to dispatch input

For each horizon step the model emits the four quantile levels; each is inverted from `asinh` to dollars and the step's values are sorted to repair any quantile crossing. The single price path the LP consumes is the quantile-integrated **mean**, $E[p] \approx \sum w_\tau Q(\tau)$, where the weights follow the piecewise-constant quantile-function rule (each quantile carries the probability mass between the midpoints of its neighbouring levels, weights summing to 1):

```

levels tau = [0.05, 0.50, 0.90, 0.98]
weights w   = [0.275, 0.425, 0.240, 0.060] (midpoint rule, sum = 1)
point price = w . sort(sinh(quantile_predictions))

```

Using the mean rather than the median is essential: revenue is linear in price and NEM prices are heavily right-skewed, so the median systematically under-values the paying tail. Probabilistic dispatch modes that consume the full fan (per-quantile scenario LPs fused by robust / expected-value / CVaR rules, and a joint mean-CVaR Rockafellar-Uryasev LP) exist in the library but did not generalise across regimes (experiment_log Entry 033) and are not on the path reported here.

Appendix B — Reproducibility pointers

Source: `src/grian/sim/{mpc,lp,oracle,models,features}.py`. Harness: `scripts/testbed.py` (build/replay fans), `scripts/run_common_eval.py` (point matrix). Narrative log: `outputs/experiment_log.md` Entries 030–035. Companion executed notebooks: `notebooks/11_making_mpc_work.ipynb` (the bug + fix) and `notebooks/12_forecast_conditioning.ipynb` (the forecast-conditioning analysis). The fix is backward-compatible: `observe_gate = 0` reproduces the legacy behaviour.